

人工智能 PJ1：搜索实验报告

姓名	学号	分工
陈家龙	24300980041	Q1、Q2、Q4、Q5、Q6、Q9
刘俊彦	24300980021	Q3、Q7、Q8、撰写报告

1 实验环境与课上核心算法

实验运行环境为 Windows、Python 3.12.14。测试全部完成，使用无图形界面的自动评分器。核心实现位于 `search.py`、`searchAgents.py` 和 `multiAgents.py`。

1.1 Q1：广度优先搜索

`breadthFirstSearch` 使用 `util.Queue` 保存“状态、到达该状态的动作列表”。每次取出队首，先检查目标；首次扩展该状态时加入已访问集合 `visited`，再按原有顺序加入合法后继及其动作路径。队列中可能存在重复状态，但同一状态不会重复扩展。队列按路径深度递增弹出，因此在本实验单位步长代价的迷宫中，首次找到的目标路径步数最少；搜索失败时返回空列表。

1.2 Q2：A* 搜索

`aStarSearch` 使用 `util.PriorityQueue`，以

$$f(s) = g(s) + h(s)$$

为优先级。队列元素保存状态、动作路径和累计路径代价 g ，用 `best_cost` 记录各状态已发现的最小 g 值。后继只有在新路径代价严格更低时才更新记录并入队；即使状态曾被扩展，也允许通过更短路径重新扩展。弹出时先跳过代价高于当前记录的旧条目，再检查目标并生成后继。因此，在启发函数可采纳且目标值为 0 时，不要求一致性也能保持最优性。 $h \equiv 0$ 时，该实现等价于一致代价搜索；位置搜索使用曼哈顿距离启发式。

1.3 Q3：极小极大搜索

`MinimaxAgent` 按智能体索引依次递归：勇士为 0 号，取后继价值最大值；每条恶龙各占一个最小化层，取后继价值最小值。一层搜索深度表示勇士与全部恶龙各行动一次，只在索引循环回勇士时增加深度。胜利、失败或达到指定深度时，调用 `self.evaluationFunction` 评价状态。根节点选择价值最高的合法动作，价值相同时保留先遇到的动作。

2 课后实验实现

2.1 Q4：四角问题建模

`CornersProblem` 将状态表示为

$$s = (p, (b_1, b_2, b_3, b_4)), \quad b_i \in \{\text{False}, \text{True}\},$$

其中 p 是勇士位置， b_i 表示第 i 个角落是否已访问。起点若处于角落，对应标记初始化为真；四个标记全部为真时达到目标。后继只包含不穿墙的上下左右移动，步长代价为 1，进入角落时更新对应标记。标记使用不可变元组，各搜索状态相互独立，不包含恶龙等无关信息。使用课上完成的 BFS 可直接搜索这个状态空间，得到经过全部角落的最短路径。

2.2 Q5: 四角启发函数

Q5、Q6 均按作业要求构造非负启发函数，并在目标状态返回 0。

设 R 为未访问角落集合， d_M 为曼哈顿距离。`cornersHeuristic` 枚举 R 的全部排列，取忽略墙壁时访问剩余角落的最短距离和：

$$h_C(p, R) = \begin{cases} 0, & R = \emptyset, \\ \min_{\pi \in \text{Perm}(R)} \left[d_M(p, \pi_1) + \sum_{i=1}^{|R|-1} d_M(\pi_i, \pi_{i+1}) \right], & R \neq \emptyset. \end{cases}$$

最多枚举 $4! = 24$ 种顺序。任意真实完成路径都对应一种角落访问顺序，且每段真实距离不小于曼哈顿距离，因此 h_C 不高估剩余代价。对一步后继 s' ，把该步接在 s' 的最优排列之前；若该步到达未访角落，则先访问该角落。这构成 s 的一个候选方案，故 $h_C(s) \leq 1 + h_C(s')$ 。因此该启发式可采纳且一致。

2.3 Q6: 全金币启发函数

`CoinSearchProblem` 的状态由勇士位置 p 与剩余金币网格组成，后继复制网格并清除进入位置的金币。设 F 为剩余金币集合，`coinHeuristic` 取到最远金币的真实迷宫距离：

$$h_F(p, F) = \begin{cases} 0, & F = \emptyset, \\ \max_{c \in F} d_{\text{maze}}(p, c), & F \neq \emptyset. \end{cases}$$

`mazeDistance` 通过 BFS 求距离，以排序后的两个端点为键，对称缓存于对象的 `heuristicInfo` 中；已计算距离在后续查询中可直接复用。

收集全部金币必然到达最远金币，故 h_F 是真实剩余代价的下界。对未被吃掉的金币 c ，最短路径距离满足 $d_{\text{maze}}(p, c) \leq 1 + d_{\text{maze}}(p', c)$ ；若本步吃掉金币，其原距离为 1，同样不超过 $1 + h_F(s')$ 。对各金币取最大值即可得到 $h_F(s) \leq 1 + h_F(s')$ ，因此该启发式可采纳且一致。

2.4 Q7: Alpha-Beta 剪枝

`AlphaBetaAgent` 沿用 Minimax 的轮次与深度定义，并传递祖先的 α 、 β 边界。最大化节点随已搜索子节点更新下界，当前价值严格大于 β 时剪枝；最小化节点更新上界，当前价值严格小于 α 时剪枝。按 `getLegalActions` 的原顺序逐个生成后继，不预先生成全部子节点，也不在相等时剪枝，以符合评分器对节点生成数量的要求。剪枝跳过不影响祖先决策的分支，保留 Minimax 的最优值。

2.5 Q8: 期望极大化

`ExpectimaxAgent` 在勇士节点仍取最大值，在恶龙节点假设每个合法动作等概率发生，使用

$$V(s) = \frac{1}{|A(s)|} \sum_{a \in A(s)} V(\text{Succ}(s, a))$$

代替最小值；多条恶龙按索引依次形成概率层。终止条件和深度计算与 Minimax 相同。Minimax 假设恶龙总选择对勇士最不利的动作，因此更保守；Expectimax 根据随机恶龙的平均结果决策，可能接受存在失败风险但期望收益更高的路线。

2.6 Q9: 改进状态评价

`betterEvaluationFunction` 评价当前状态。设 S 为游戏分数, d_c 、 d_s 分别为最近金币、宝剑的曼哈顿距离, d_i 为第 i 条恶龙的曼哈顿距离, t_i 为其剩余虚弱时间。非终局评价为

$$E_0(s) = S + C(s) + B(s) + \sum_i \frac{w(t_i)}{d_i + 1},$$

其中, 存在金币时 $C(s) = 1/(d_c + 1)$, 否则为 0; 存在宝剑时 $B(s) = 7/(d_s + 1)$, 否则为 0。恶龙权重为

$$w(t) = \begin{cases} -10, & t = 0, \\ 5, & t > 5, \\ -2, & 0 < t \leq 5. \end{cases} \quad E(s) = \begin{cases} S + 100000, & \text{胜利,} \\ S - 100000, & \text{失败,} \\ E_0(s), & \text{非终局.} \end{cases}$$

游戏分数提供已取得收益的信号; 距离奖励引导接近金币与宝剑; 正常恶龙受到较强惩罚, 虚弱时间充足时鼓励追击, 虚弱即将结束时改为轻度回避。分母加 1 避免距离为 0 时除零, 终局的大幅加减分优先区分胜负。评价使用曼哈顿距离以控制计算开销, 其距离信号可能忽略墙壁绕行, 是当前设计的局限。

3 自动评分结果

在仓库根目录、Python 3.12.14 环境下执行:

```
python autograder.py --no-graphics
```

现有自动评分用例全部通过, 结果如下。表中的“扩展”指对应外层搜索问题的扩展节点数。Q9 使用 `smallClassic`、深度为 2 的 `ExpectimaxAgent` 和 `evalFn=better`, 与一条随机恶龙对战; 随机种子设为 0, 连续运行 10 局。决策时间为平均每局勇士累计思考时间。

题目	完成情况与主要验证结果	得分
Q1	BFS 已实现; <code>mediumMaze</code> : 68 步, 扩展 269 个节点	3/3
Q2	A* 已实现; 同图使用曼哈顿启发式: 68 步, 扩展 221 个节点	3/3
Q3	Minimax 已实现; 通过仓库现有评分用例	5/5
Q4	四角状态建模完成; <code>tinyCorners</code> 最短路径为 28 步	3/3
Q5	四角启发式完成; <code>mediumCorners</code> : 106 步, 扩展 741 个节点	3/3
Q6	金币启发式完成; <code>trickySearch</code> 扩展 4137 个节点	4/4
Q7	Alpha-Beta 已实现; 通过动作、深度及节点生成检查	5/5
Q8	Expectimax 已实现; 通过概率节点、多恶龙及深度检查	5/5
Q9	改进评价完成; 10/10 局获胜, 平均得分 1075.0, 平均决策时间 0.763 秒, 均未超时	6/6
合计	课上 Q1–Q3: 11/11; 课后 Q4–Q9: 26/26	37/37